CO326 Industrial Digital Twin & Cyber-Physical Security Project Report

Group 19

Smart Humidity Monitoring Digital Twin

GitHub Repository: https://github.com/cepdnaclk/e20-co326-Smart-Humidity-Monitoring-Digital-Twin

Group Members

Name	Registration Number
Tharusha Haththella	E/20/133
Sachindu Premasiri	E/20/305
Nimesha Somathilaka	E/20/381
Devin Sriyarathna	E/20/385

1. Introduction

This project implements a Smart Humidity Monitoring Digital Twin for an industrial storage environment. Humidity is an important parameter in pharmaceutical stores, food storage rooms, electronics storage areas, textile warehouses, and cleanroom environments.

Abnormal humidity can damage products, reduce quality, and create unsafe storage conditions. Therefore, the system continuously monitors humidity and temperature, detects abnormal conditions at the edge, stores time-series data, and visualizes the live system state using a Grafana dashboard.

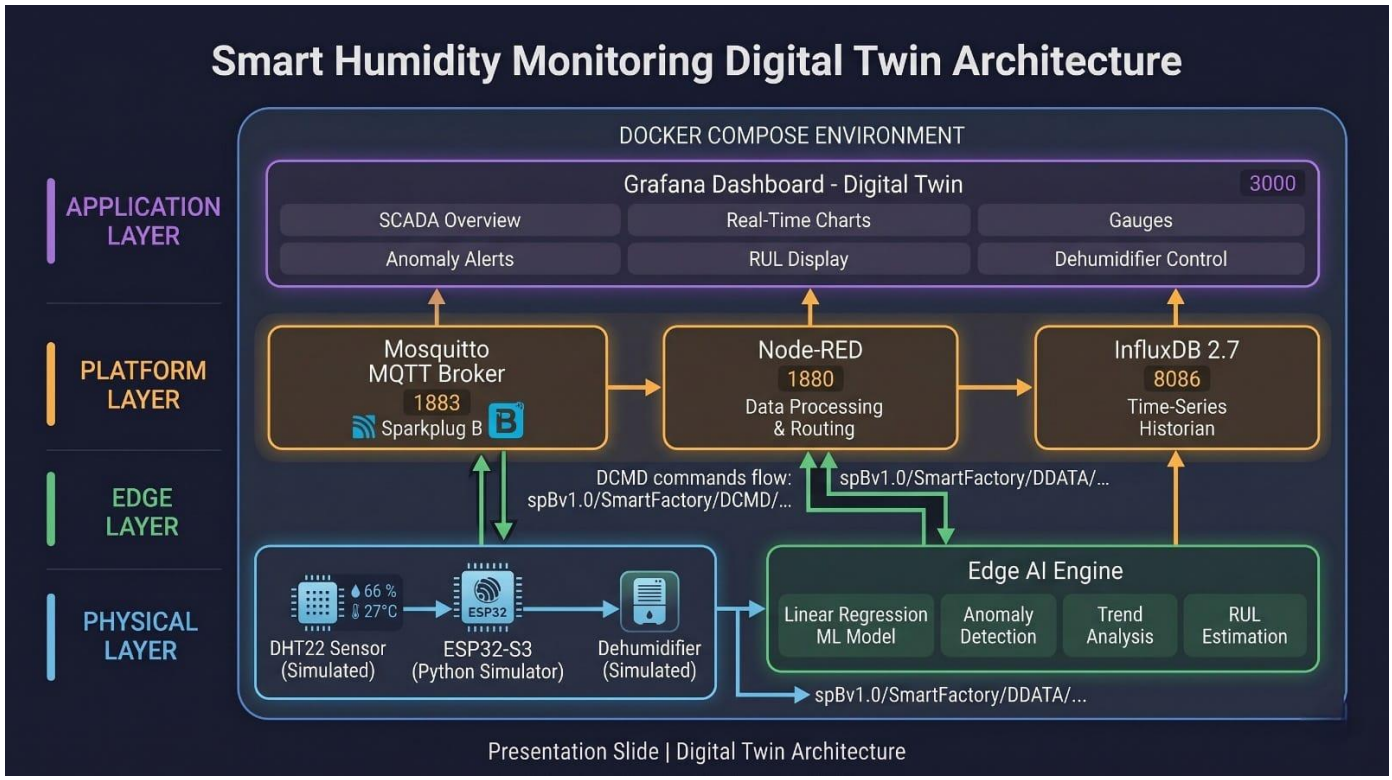
The system follows an edge-to-cloud Industrial IoT pipeline using ESP32-S3, MQTT, Node-RED, InfluxDB, Grafana, and Docker Compose. During testing, a simulated DHT22-style data source and a simulated dehumidifier state were used to demonstrate the complete Digital Twin workflow.

2. Objectives

- Collect humidity and temperature data using an ESP32-S3 edge device or simulator.
- Publish structured telemetry data using MQTT.
- Process incoming data using Node-RED.
- Store time-series telemetry in InfluxDB.
- Visualize live and historical values using Grafana.
- Detect abnormal humidity behavior using edge-level anomaly detection.
- Estimate system health and RUL-related indicators using trend analysis.
- Demonstrate Digital Twin behavior using dashboard visualization and command topics.
- Document the system using industrial implementation standards.

3. System Overview

The Smart Humidity Monitoring Digital Twin contains an edge device layer, MQTT messaging layer, flow-processing layer, historian database layer, and visualization layer. The implementation is containerized using Docker Compose for repeatable local deployment.



Component	Technology / Tool	Purpose
Humidity and temperature sensing	DHT22 sensor / simulated DHT22 stream	Generates humidity and temperature readings for monitoring
Edge device	ESP32-S3 / Python simulator	Runs edge logic, ML prediction, anomaly detection, trend analysis and MQTT publishing
Message broker	Mosquitto MQTT :1883	Transfers telemetry, alerts, status and command messages
Flow logic	Node-RED :1880	Parses MQTT payloads, routes data, generates alerts and writes to InfluxDB
Historian	InfluxDB 2.7 :8086	Stores time-series data for Flux queries
Application dashboard	Grafana :3000	Provides SCADA-style Digital Twin visualization and operator view

Figure 1: High-level architecture of the Smart Humidity Monitoring Digital Twin.

3.1 High-Level Data Flow

```
DHT22 sensor / simulation
        -> ESP32-S3 or edge simulator
        -> Mosquitto MQTT broker
        -> Node-RED processing and routing
        -> InfluxDB time-series storage
        -> Grafana Digital Twin dashboard
```

4. Four-Layer IIoT Architecture

4.1 Physical Layer

The physical layer represents the humidity-sensitive industrial environment. The sensing element is a DHT22-style humidity and temperature source. A dehumidifier is represented as a simulated actuator state so that the Digital Twin can show the control status without depending on extra physical actuation hardware.

4.2 Edge Layer

The edge layer runs sensor reading, short-term history buffering, linear-regression-based humidity prediction, anomaly scoring, trend detection and MQTT publishing. The ESP32 firmware reads the DHT22 sensor from GPIO27, maintains five previous readings, and publishes a JSON telemetry message every two seconds.

4.3 Platform Layer

The platform layer runs inside the Docker Compose environment. Mosquitto handles MQTT messaging, Node-RED performs message parsing and routing, and InfluxDB stores telemetry as time-series data. Node-RED also separates data ingestion, alert processing, RUL estimation, device status and command handling into different flow sections.

4.4 Application Layer

Grafana provides the operator-facing Digital Twin view. It displays live humidity, temperature, anomaly score, RUL estimate, device status, dehumidifier state and time-series trends. The dashboard therefore acts as the main SCADA-style interface for the project demonstration.

5. MQTT Topic Hierarchy

The system uses a structured MQTT topic hierarchy inspired by Sparkplug B and Unified Namespace principles. The topic structure separates telemetry, command, alert, RUL and status messages.

Topic	Purpose
spBv1.0/SmartFactory/DDATA/HumidityMonitoring/humidityTwin01	Live humidity, temperature, prediction, anomaly score and trend data
spBv1.0/SmartFactory/DCMD/HumidityMonitoring/humidityTwin01	Dashboard-to-device command path for dehumidifier control
spBv1.0/SmartFactory/DDATA/HumidityMonitoring/humidityTwin01/anomaly	Warning and critical anomaly alert messages
spBv1.0/SmartFactory/DDATA/HumidityMonitoring/humidityTwin01/rul	RUL or health trend estimation output
spBv1.0/SmartFactory/DSTATE/HumidityMonitoring/humidityTwin01/status	Device state and LWT-based online/offline indication

5.1 Example Telemetry Payload

```
{
  "timestamp": "2026-05-01T00:00:00Z",
  "device_id": "humidityTwin01",
  "humidity": 46.60,
  "temperature": 25.20,
  "ml_prediction": 44.90,
  "anomaly_score": 1.63,
  "anomaly_status": "NORMAL",
  "trend": "STABLE",
  "status": "ONLINE"
}
```

6. Edge AI and Anomaly Detection

The edge anomaly detection logic is lightweight and suitable for execution on the ESP32. The provided firmware uses a trained linear regression model to predict humidity from recent humidity and temperature history. The anomaly score is calculated as the absolute difference between the actual humidity and the predicted humidity.

Firmware / Model Item	Implementation Detail
Sensor library	DHT sensor library and Adafruit Unified Sensor
MQTT library	PubSubClient
Sensor pin	GPIO27
Sampling interval	2 seconds
History window	5 readings
ML model	Linear regression with lag features
Anomaly score	abs(current humidity - ML prediction)
Warning threshold	anomaly score > 3.0 RH percentage points
Critical threshold	anomaly score > 5.0 RH percentage points
Trend rule	Increasing/decreasing if humidity changes by more than 0.2 RH percentage points

6.1 Edge Decision Logic

```
IF history_count < 5:
    status = INITIALIZING
ELSE:
    prediction = linear_regression(lagged humidity and temperature)
    anomaly_score = abs(current_humidity - prediction)
    IF anomaly_score > 5.0:
        status = CRITICAL
    ELSE IF anomaly_score > 3.0:
        status = WARNING
    ELSE:
        status = NORMAL
```

7. Node-RED and InfluxDB Design

Node-RED is the edge-logic and orchestration layer. It subscribes to telemetry and status topics, parses JSON messages, converts values into the required InfluxDB format, generates alert messages, estimates RUL-related values, and publishes command messages for the simulated dehumidifier.

- Data ingestion pipeline: receives MQTT telemetry, parses fields and writes humidity, temperature, prediction and anomaly score to InfluxDB.
- Alert pipeline: filters abnormal values and publishes warning or critical alerts.
- RUL estimation pipeline: calculates trend-based RUL indicators and stores/publishes the result.
- Status and LWT pipeline: tracks online/offline device state.
- Bidirectional control pipeline: sends dashboard commands through the DCMD topic.

Parameter	Value
InfluxDB version	2.7
Organization	co326_org
Bucket	co326_bucket
Measurement	humidity_twin
MQTT broker	Mosquitto Broker

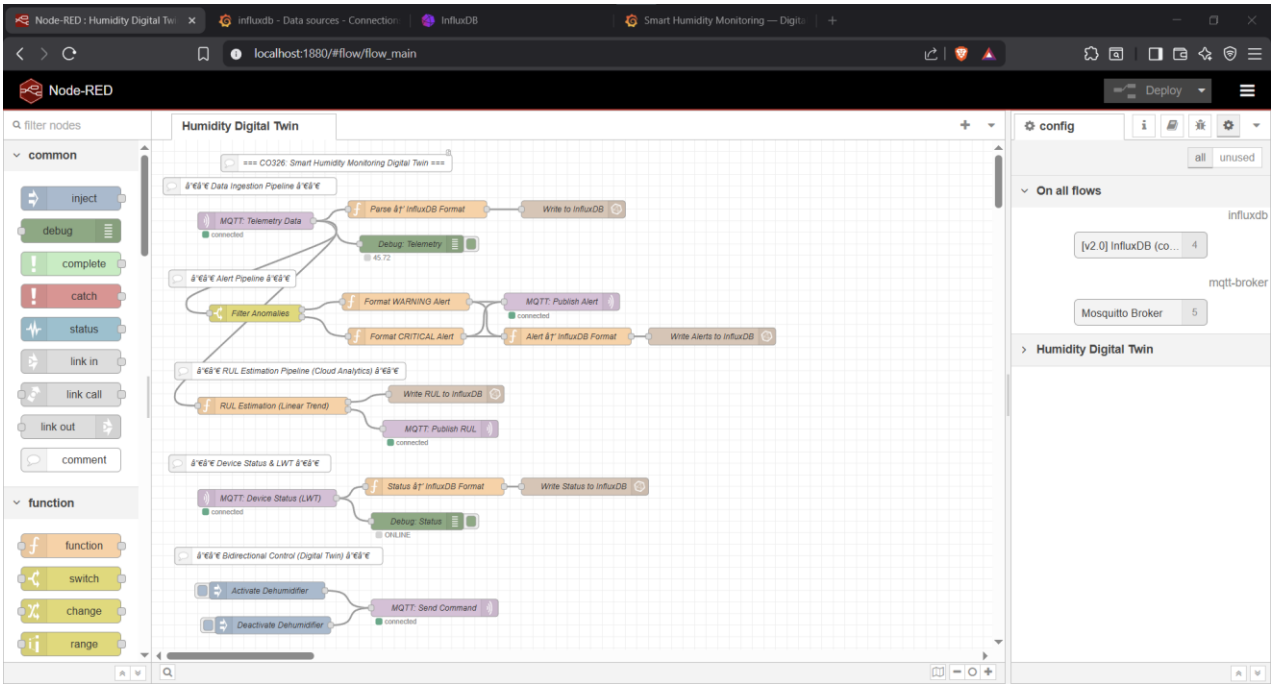


Figure 2: Node-RED flow showing data ingestion, alerting, RUL estimation, device status and command handling.

8. Grafana Dashboard

Grafana is used as the Digital Twin dashboard. It provides the operator view for monitoring the current system state, historical trends, anomaly score, RUL estimate and dehumidifier status. The dashboard is connected to InfluxDB and refreshed in near real time.

- System overview cards for device status, current humidity, current temperature, anomaly score, RUL estimate and dehumidifier status.
- Real-time gauges for humidity and temperature.
- Time-series panels for humidity, ML prediction and temperature.
- Edge AI analytics panels for anomaly score and RUL estimation.
- Dashboard-level status display for the simulated dehumidifier.

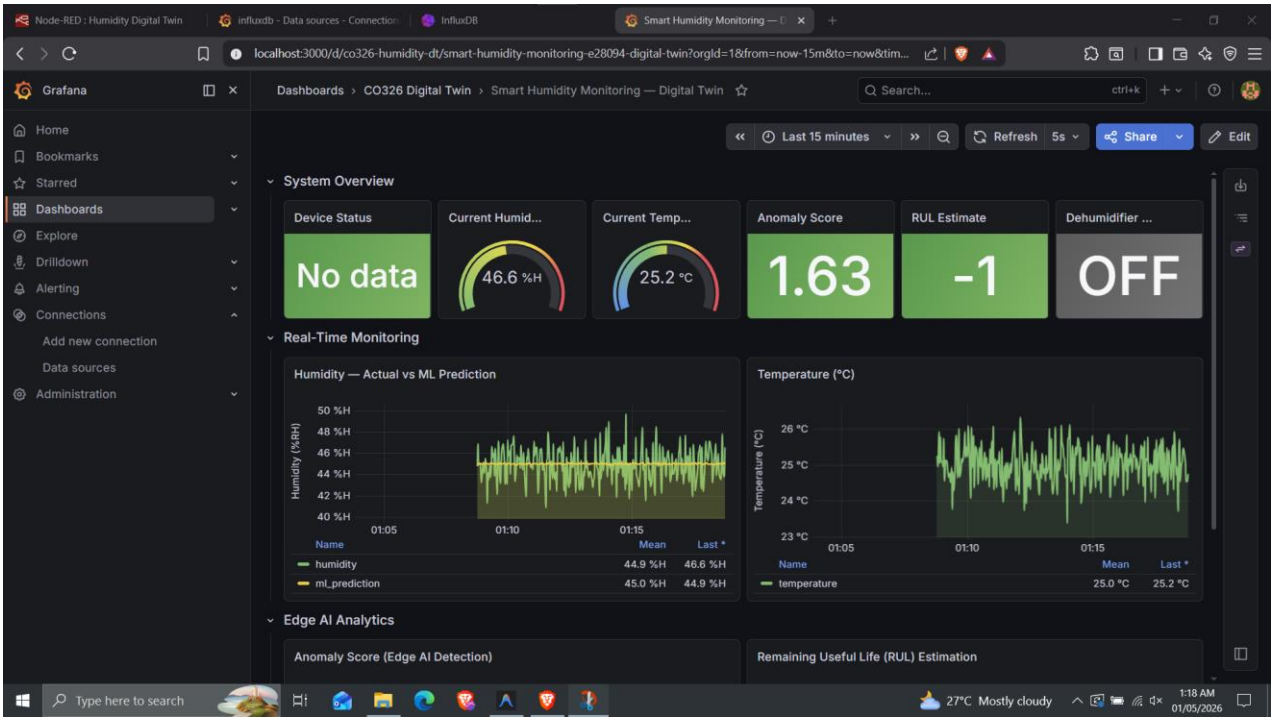


Figure 3: Grafana system overview and real-time monitoring panels.

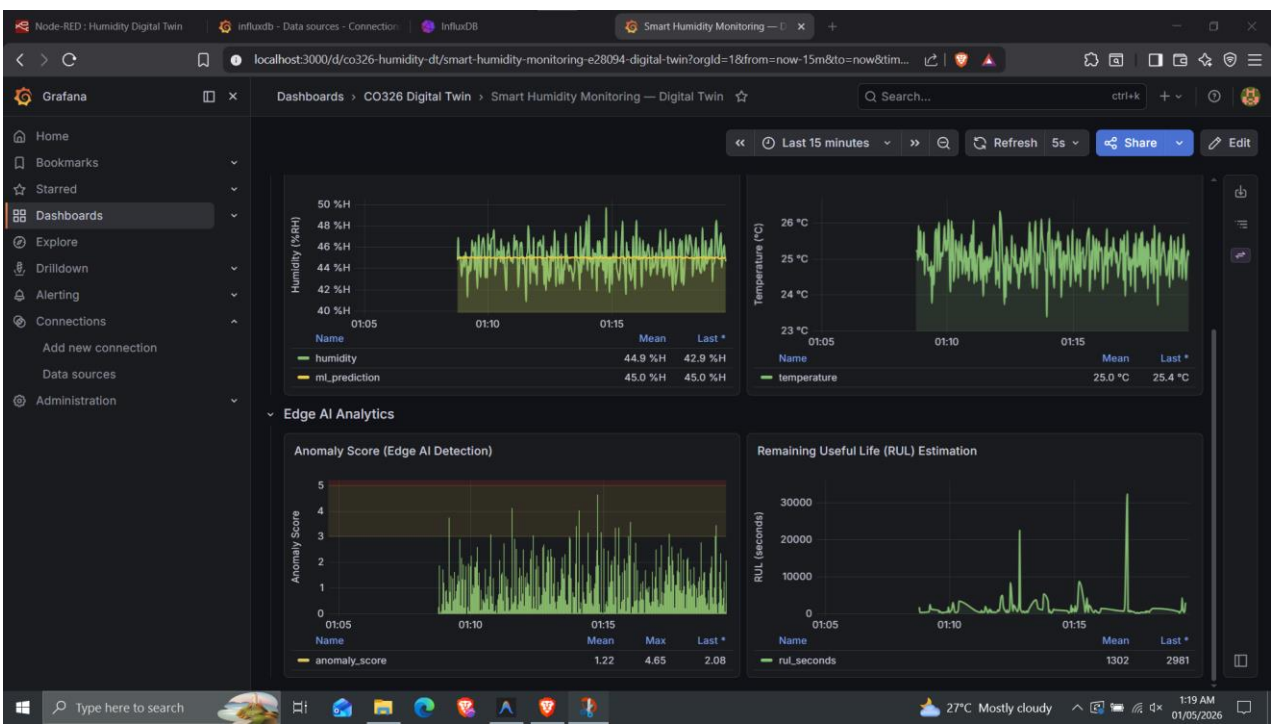


Figure 4: Grafana trend dashboard with humidity, temperature, anomaly score and RUL panels.

9. Simulation Mode

Simulation mode was used to test the complete pipeline before depending on physical environmental changes. The simulator produces DHT22-like humidity and temperature readings, allowing the team to demonstrate normal operation, trend changes, alerts and dashboard updates.

Scenario	Expected Output
Normal humidity variation	Telemetry updates remain in NORMAL status
Humidity rises above expected trend	Anomaly score increases and status changes to WARNING if threshold is exceeded
Large deviation from ML prediction	Status changes to CRITICAL
Device online message	Grafana and Node-RED display ONLINE status
Dehumidifier command	Command is published through DCMD and simulated state changes

10. Cybersecurity and Reliability

10.1 Cybersecurity Features

- MQTT username/password authentication can be enabled in Mosquitto configuration.
- Controlled topic access can be enforced using MQTT ACL rules.
- Credentials should be stored separately from the main source code and not committed to GitHub.
- Dashboard access should be protected with Grafana authentication.
- For production deployment, MQTT TLS encryption should be enabled.

10.2 Reliability Features

- MQTT Last Will and Testament supports automatic offline detection.
- The edge publisher reconnects to Wi-Fi and MQTT when connection drops occur.
- All telemetry includes timestamps for traceability.
- Docker restart policies can recover platform services after a failure.
- InfluxDB stores historical telemetry for later analysis and dashboard queries.

11. Testing and Results

The following table summarizes the verification status based on the current implementation screenshots and available firmware.

Test Case	Expected Result	Actual Result
Docker stack startup	Grafana, Node-RED, Mosquitto and InfluxDB run successfully	Verified through local browser access to Grafana :3000 and Node-RED :1880
ESP32 / simulator telemetry	Humidity and temperature readings are generated	Verified through live Grafana values around 46 percent RH and 25 deg C
MQTT publishing	Telemetry reaches Mosquitto broker	Verified by connected MQTT nodes in Node-RED
Node-RED processing	Telemetry is parsed, routed and written to InfluxDB	Verified by data ingestion, alert, RUL and status flows
InfluxDB storage	Data is stored as time-series measurements	Verified by Grafana panels querying historical values
Grafana visualization	Dashboard displays live and historical data	Verified with gauges, trend panels and analytics panels
Anomaly detection	Anomaly score updates on dashboard	Verified with anomaly score panel and current score card
RUL estimation	RUL estimate is generated and visualized	Verified by RUL panel and RUL card
Device status / LWT	Online/offline state can be monitored	Status pipeline implemented; final disconnect evidence can be added after demo testing
Dehumidifier control	Command path exists through DCMD topic	Simulated dehumidifier state is shown as OFF on dashboard

Current dashboard evidence shows that the data pipeline is active: the humidity and temperature panels update over time, the anomaly score is generated by the edge analytics layer, and the RUL panel receives values from the analytics pipeline.

12. Discussion

The implemented system demonstrates a complete edge-to-cloud Industrial IoT monitoring pipeline. MQTT provides lightweight publish/subscribe communication, Node-RED enables flexible data processing and orchestration, InfluxDB stores time-series measurements, and Grafana provides a clear SCADA-style dashboard for visualization.

The edge AI approach is intentionally lightweight. Linear regression with lag features is suitable for low-resource devices and allows the system to compare the actual humidity value against an expected value. This produces an anomaly score that is easy to interpret during demonstrations.

The use of simulation makes it possible to test normal and abnormal humidity behavior repeatedly. This is useful because real environmental humidity changes may be slow and difficult to reproduce within a short demonstration period.

13. Limitations and Future Improvements

13.1 Limitations

- The current prototype focuses on monitoring, analytics and simulated actuator state rather than full industrial closed-loop control.
- The anomaly model is lightweight and uses a small set of lag-based features.
- The implementation is tested in a local Docker environment.
- Sensor accuracy depends on the selected humidity sensor and calibration quality.
- Final hardware validation results should be added after extended testing.

13.2 Future Improvements

- Use an industrial-grade humidity sensor for improved accuracy and robustness.
- Add multiple sensing zones for warehouse or cleanroom monitoring.
- Enable MQTT TLS encryption for secure deployment.
- Add alert notifications through email, Telegram or dashboard alerting.
- Improve the RUL model using longer historical datasets.
- Deploy the dashboard on a cloud or campus server for remote monitoring.

14. Conclusion

This project implemented a Smart Humidity Monitoring Digital Twin using ESP32-S3, MQTT, Node-RED, InfluxDB, Grafana and Docker Compose. The system collects humidity and temperature data, performs edge-level ML prediction and anomaly detection, stores data in a time-series database, and visualizes the live system state through a SCADA-style Grafana dashboard.

The final prototype demonstrates the main concepts of Industrial IoT and Digital Twin design, including edge data acquisition, MQTT communication, flow-based processing, historian storage, dashboard visualization, anomaly detection, RUL trend estimation, status monitoring and simulated bidirectional control.

15. Contributions

Name	Registration Number	Technical Contribution	Report Contribution
Tharusha Haththella	E/20/133	Set up the Docker-based deployment environment, including Mosquitto MQTT broker, Node-RED, InfluxDB, and Grafana services. Configured the containerized platform, supported service connectivity, verified Docker Compose execution, and helped ensure the backend IIoT services were running reliably.	Wrote the Docker deployment section, platform setup explanation, service configuration description, container connectivity explanation, and related deployment documentation.
Sachindu Premasiri	E/20/305	Handled system integration, cybersecurity and reliability, GitHub documentation organization, testing coordination, final report compilation, and supported ML integration into the end-to-end monitoring pipeline.	Wrote the system overview, MQTT topic hierarchy, Node-RED flow explanation, dashboard explanation, ML integration explanation, testing/results, discussion, and final report formatting.
Nimesha Somathilaka	E/20/381	Developed the ML-related logic for humidity prediction, anomaly detection, trend analysis, and Remaining Useful Life estimation. Tuned the abnormal humidity detection logic, supported AI output interpretation, and helped validate the ML results shown in the dashboard.	Wrote the Edge AI section, anomaly detection explanation, ML model description, humidity prediction logic, trend analysis, RUL estimation section, and AI result interpretation.
Devin Sriyarathna	E/20/385	Worked on hardware setup, soft sensor fine-tuning, and the connection between the hardware layer and the AI model. Supported sensor data preparation, hardware-side validation, and integration of physical/simulated readings into the AI-based monitoring pipeline.	Wrote the hardware setup section, soft sensor fine-tuning explanation, hardware-to-AI model connection, perception layer, sensor data preparation, and hardware testing documentation.

16.References

1. OASIS, “MQTT Version 5.0,” OASIS Standard, 2019. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
2. MQTT.org, “MQTT: The Standard for IoT Messaging.” Available: https://mqtt.org/

3. Node-RED, “Node-RED: Low-code programming for event-driven applications.” Available: https://nodered.org/
4. InfluxData, “InfluxDB OSS v2 Documentation.” Available: https://docs.influxdata.com/influxdb/v2/
5. Grafana Labs, “Grafana Documentation: Data Sources.” Available: https://grafana.com/docs/grafana/latest/datasources/

17.AI Assistance Disclosure

AI tools were used to support project planning, documentation drafting, implementation guidance, and report structuring.

- ChatGPT was used to refine explanations, generate documentation templates, and organize the report content.
- Google Antigravity was used as an AI-assisted development environment to support implementation planning and code generation. All final technical decisions, implementation, testing, verification, and submitted content were reviewed and validated by the project team.

1. OpenAI, “ChatGPT.” Available: https://chatgpt.com/
2. Google, “Google Antigravity Documentation.” Available: https://antigravity.google/docs/home